

# Foundations of Markov decision process

[Markov decision process]

## 1.0 Content Block

where  $H$  is the time horizon. Compared to the previous objective, the latter one is more used in Learning Theory. A policy that maximizes the function above is called an optimal policy and is usually denoted  $\pi^*$ . A particular MDP may have multiple distinct optimal policies. Because of the Markov property, it can be shown that the optimal policy is a function of the current state, as assumed above. When  $R_a(s, s')$  is deterministic, there will always exist an optimal policy  $\pi^*$  which is deterministic as well.

## 2.0 Content Block

**Prioritized sweeping** In this variant, the steps are preferentially applied to states which are in some way important – whether based on the algorithm (there were large changes in  $V$  or  $\pi$  around those states recently) or based on use (those states are near the starting state, or otherwise of interest to the person or program using the algorithm).

## 3.0 Content Block

A Markov decision process (MDP) is a mathematical model for sequential decision making when outcomes are uncertain. It is a type of stochastic decision process, and is often solved using the methods of stochastic dynamic programming. Originating from operations research in the 1950s, MDPs have since gained recognition in a variety of fields, including ecology, economics, healthcare, telecommunications and reinforcement learning. Reinforcement learning utilizes the MDP framework to model the interaction between a learning agent and its environment. In this framework, the interaction is characterized by states, actions, and rewards. The MDP framework is designed to provide a simplified representation of key elements of artificial intelligence challenges. This modeling framework incorporates the understanding of cause and effect, the management of uncertainty and nondeterminism, and the pursuit of explicit goals. The name comes from its connection to Markov chains, a concept developed by the Russian mathematician Andrey Markov. The "Markov" in "Markov decision process" refers to the underlying structure of state transitions that still follow the Markov property. The process is called a "decision process" because it involves making decisions that influence these state transitions, extending the concept of a Markov chain into the realm of decision-making under uncertainty.

## 4.0 Content Block

$E \left[ \sum_{t=0}^{H-1} R_a(s_t, s_{t+1}) \right]$  (where we choose  $a_t = \pi(s_t)$ , i.e. actions given by the policy). And the expectation is taken over  $s_{t+1} \sim P_a(s_t, s_{t+1})$

## 5.0 Content Block

Their order depends on the variant of the algorithm; one can also do them for all states at once or state by state, and more often to some states than others. As long as no state is permanently excluded from either of the steps, the algorithm will eventually arrive at the correct solution.

## 6.0 Content Block

$\sum_{i \in S} \sum_{a \in A(i)} R(i, a) y^*(i, a) \geq \sum_{i \in S} \sum_{a \in A(i)} R(i, a) y(i, a)$

### 7.0 Content Block

Value iteration In value iteration (Bellman 1957), which is also called backward induction, the  $\pi$  function is not used; instead, the value of  $\pi(s)$  is calculated within  $V(s)$  whenever it is needed. Substituting the calculation of  $\pi(s)$  into the calculation of  $V(s)$  gives the combined step;

### 8.0 Content Block

The solution above assumes that the state  $s$  is known when action is to be taken; otherwise  $\pi(s)$  cannot be calculated. When this assumption is not true, the problem is called a partially observable Markov decision process or POMDP.

### 9.0 Content Block

$R_a(s, s')$  is 1 if the pole is up after the transition, zero otherwise. Therefore, this function only depend on  $s'$  in this specific case.

### 10.0 Content Block

Discrete space: Linear programming formulation If the state space and action space are finite, we could use linear programming to find the optimal policy, which was one of the earliest approaches applied. Here we only consider the ergodic model, which means our continuous-time MDP becomes an ergodic continuous-time Markov chain under a stationary policy. Under this assumption, although the decision maker can make a decision at any time in the current state, there is no benefit in taking multiple actions. It is better to take an action only at the time when system is transitioning from the current state to another state. Under some conditions, if our optimal value function  $V^*$  is independent of state  $i$ , we will have the following inequality:

### 11.0 Content Block

$S$  is the set of ordered tuples  $(\theta, \dot{\theta}, x, \dot{x})$  given by pole angle, angular velocity, position of the cart and its speed.

### 12.0 Content Block

Another application of MDP process in machine learning theory is called learning automata. This is also one type of reinforcement learning if the environment is stochastic. The first detail learning automata paper is surveyed by Narendra and Thathachar (1974), which were originally described explicitly as finite-state automata. Similar to reinforcement learning, a learning automata algorithm also has the advantage of solving the problem when probability or rewards are unknown. The difference between learning automata and Q-learning is that the former technique omits the memory of Q-values, but updates the action probability directly to find the learning result. Learning automata is a learning scheme with a rigorous proof of convergence. In learning automata theory, a stochastic automaton consists of:

### 13.0 Content Block

$R_a(s, s')$  is the immediate reward (or expected immediate reward) received after action  $a$  is taken to transition from state  $s$  to state  $s'$ . The reward is in general a random variable. A policy function  $\pi$  is a (potentially probabilistic) mapping from state space  $(S)$  to action space  $(A)$ .